Course Project

This is a group project. Each group will have 3 people. This project is marked out of 60 points.

Due Date: April 24, Friday, 2026, 11:59 PM

Project Overview

This project investigates how different fine-tuning strategies affect the performance and robustness of image classification models on the Food101 dataset [1]. Students will compare parameter-efficient methods with full fine-tuning using pretrained backbones such as ResNet [2] and EfficientNetV2 [3]. Models will be evaluated on both clean and corrupted test sets to assess their ability to generalize under different data distribution shifts. The project also emphasizes trade-offs among accuracy, computational cost, and model efficiency, and analyzes model behavior using explainability techniques. Teams are encouraged to take some time to familiarize themselves with the required techniques before starting the project.

Project Requirements

All teams must use the following pretrained backbone architectures, initialized with ImageNet-1K pretrained weights:

- ResNet18
- EfficientNetV2-S

Students are required to implement and compare the following training strategies:

1. Linear probing
2. Parameter-efficient fine-tuning (PEFT) methods, including:
    1. LoRA [4]
    2. Task Specific Adapters [5]
    3. BatchNorm tuning [6]
3. (Optional) Full fine-tuning of all model parameters[1]

Each team must establish a baseline model using linear probing with no additional data augmentation or regularization beyond default optimizer settings. A custom model must also be developed, incorporating improved training or inference strategies such as, but not limited to, data augmentation, regularization, or alternative fine-tuning approaches. All training must be performed using only the train-split of the dataset. Test sets, including any transformed variants, must be used strictly for evaluation.

All models must be evaluated on both the clean test set and the transformed test variants provided for robustness analysis. All teams must report:

1- Performance on the clean test set
2- Robustness scores on each test set variant
3- Aggregate performance across all variants, if desired, for summary analysis

[1] If omitted, teams must explain why and note any attempts using free public GPUs (e.g., Google Colab, Kaggle). No penalty will be applied if effort was made but resources were insufficient. Discuss with the TA for more information.

For analysis, teams must produce visualizations showing trade-offs between performance and model efficiency, such as bubble plots of performance vs FLOPs[2] or wall-clock time, with bubble size proportional to trainable parameter count. Similar plots should be created for inference speed vs performance.

Finally, teams must analyze and interpret why different model variants fail on specific test variants. Techniques such as Grad-CAM [7] or other explainable AI methods must be used to visualize which regions of an image the model focuses on when making predictions. The goal is to provide insight into the model behavior, for example:

- Whether the model relies on specific image regions
- Differences in attention patterns between linear probes, PEFT methods, and full fine-tuning

All the test set variants can be found in the following link (~4.07 GB):
https://drive.google.com/file/d/1MvkUSd2ESqBzXgObUYBBxOH6kMFLDP5B/view?usp=sharing

Bonus

Teams may extend the project by exploring recent methods, such as Vision Transformers or newer PEFT techniques and alternative classifiers, from 2024 or later. These extensions are optional.

Recommended Repositories

Below is a list of helpful GitHub repositories that you may use for reference or implementation, but are not required to use:

- LoRA: https://github.com/microsoft/LoRA/tree/main
- Task Specific Adapters:[3] https://github.com/VICO-UoE/URL/blob/master/models/tsa.py
- Grad-CAM: https://github.com/jacobgil/pytorch-grad-cam
- FLOPs calculation: https://github.com/MrYxJ/calculate-flops.pytorch

Deliverables

The deliverables are (1) a project report, (2) an .ipynb file, and (3) a project presentation compressed into a single ZIP file. *Do not include the dataset in your submission.*

The project report is submitted as a single PDF file (12-point font and 1.5-spaced). Regarding how to write a high-quality report to show your project achievements, the following guidelines are provided for your reference. The goal of preparing a project report is to demonstrate more details about your project ideas, goals, methods, and results. It should include the following sections:

- Summary (200-250 words): A concise and factual summary of the project conducted, including motivation for the project, a brief overview of the proposed method, and the outcomes.
- Introduction (1-2 pages): A brief overview of the background and context of the topic. This section motivates the project and gives an overview of the proposed method. The section

[2] FLOPs stand for Floating Point Operations.

[3] You do not have to use the Pre-classifier Alignment part of the codebase. Discuss with the TA in case of difficulties implementing the algorithm.

should also discuss recent and relevant work in the area. You can include relevant citations to prior work on your topic and describe how they are relevant. The citations are not limited to research publications or book chapters, but can also include website URLs.

- Methodology (3-5 pages): A detailed description of how the project was conducted. Describe the proposed method to the stated problem, as clearly as possible. Include any necessary diagrams and/or figures to help explain the method.
- Experiments (3-5 pages): A detailed description of experimental design, results, analysis, and discussion. Describe the experimental setup and considerations. This section includes the results and a detailed analysis of the results and choices made.
- Conclusion and Discussion for Future Work (1-2 pages): A general summary of the main ideas and insights in the project conducted. Take a step back and evaluate the pros and cons of your method from a higher level. List things that can be done towards the future improvement of the project (if applicable).
- References (≤ 1 page): A list of the sources of the cited information.
- Contributions of group members (if applicable).

The .ipynb file contains the scripts (including the results after running) used for result replication. It should be accessible, understandable, and replicable. The project presentation highlights and summarizes the project conducted using slides. It should be planned for 8 to 10 minutes.

Marking Rubric

Presentation (10 points)
Evaluation metrics: Clearly demonstrate the project goals, progress/completeness, and the obtained important results, so that the audience can overall understand your project goals, methods, results, and contributions without reading the report.

Report (50 points)

(1) Project and experiments completeness: 20 points
   - The code in .ipynb files can be smoothly run by the TA without debugging.
   - Enough experiments to convince the reader of the effectiveness of your proposed model.

(2) Project analysis: 20 points
   - Your project should include your own analysis (quantitative or qualitative), rather than simply restating/redemonstrating performance metrics. You must extend/expand upon existing works and demonstrate your understanding of the methods and any modifications that can be applied.
   - You must explain all the different trade-offs when training such models and why different choices were made.

(3) Overall quality: 10 points
   - Correctness: The content should be accurate, truthful, and properly cited.
   - Clearness: Students should try to use plain language to explain the concepts and avoid using uncommon words or GPT-like words.

Plagiarism from the internet or peers is not allowed.

References

[1] L. Bossard, M. Guillaumin, and L. Van Gool, "Food-101 – Mining Discriminative Components with Random Forests," *in Proc. European Conference on Computer Vision (ECCV), ser. Lecture Notes in Computer Science*, vol. 8694, 2014, pp. 446–461, doi:10.1007/978-3-319-10599-4_29.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778, doi:10.1109/cvpr.2016.90.

[3] M. Tan and Q. V. Le, "EfficientNetV2: Smaller models and faster training," in *Proc. Int. Conf. on Machine Learning (ICML)*, 2021, pp. 10096–10106.

[4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2022.

[5] WH. Li, X. Liu, and H. Bilen, "Cross-domain few-shot learning with task-specific adapters," in *Proc. of the IEEE Conf. on Computer Vision and Pattern Recognition*, 2022, pp. 7161–7170.

[6] J. Frankle, D. J. Schwab, and A. S. Morcos, "Training BatchNorm and Only BatchNorm: On the Expressive Power of Random Features in CNNs," *in Proc. Int. Conf. on Learning Representations (ICLR),* 2021.

[7] R. R. Selvaraju *et al.*, "Grad-cam: Visual explanations from deep networks via gradient-based localization,"in *Proc. IEEE Int. Conf. on Computer Vision (ICCV)*, 2017, pp. 618–626, doi:10.1109/iccv.2017.74.